

V8のArray.prototype.flatを 最大約20倍速くするまでと、 巨大OSSへの大規模コミットの道のり

西 悠太 / 株式会社ダイニー

自己紹介

西 悠太 (Nishi Yuta)

21歳(重要、important)

V8 Contributor

Platform Engineer at Dinii Inc.

TSKaigi Staff

TypeScriptが好きです

V8にパッチを送るのも好きです

あと野球も好きです

@riya-amemiya



そもそも何でこのパッチを出したのか

ある日、Xのタイムラインに「JSCのArray.prototype.flatが速くなる」という投稿が流れてきました。

実装を読むと同じ発想がV8にも適用できそうに見えました

約1ヶ月後、私のパッチはV8 14.7にマージされました。

**Chrome、Node.js、Deno、V8を積む
あらゆるランタイムでflatが最大約20倍速くなります。**

最適化メインで話します

- なぜ従来のflatは遅かったのか
- メモリ割り当てを1回に減らす2パス方式
- holeを詰めるflat特有の仕様との格闘
- V8独自言語Torqueでの実装

上記について順を追って解説します。

v8-devでの提案からGerritレビュー、マージまでの流れも紹介します。

なぜ従来のflatは
遅かったのか

flatの仕様を理解する

```
> [[1, 2], [3, 4]].flat()  
[ 1, 2, 3, 4 ]  
> [[[1]], [[2]]].flat(2)  
[ 1, 2 ]
```

- `flat(depth)` はネストした配列を `depth` の深さまで平坦化
- `depth` を省略すると1

holeの扱い

JavaScriptの配列では、要素が存在しないインデックスのことをholeと呼びます。

```
> [1, , 3]  
[ 1, <1 empty item>, 3 ]
```

holeは `undefined` とは異なり、プロパティ自体が存在しない状態です。

従来のflatの実装

1. 空の結果配列を作る
2. ソース配列を先頭から走査する
3. 要素が配列なら、再帰で中へ降りる
4. それ以外なら、結果配列の末尾に1個追加する

どこが遅かったのか

- 結果配列が空から伸びるので、容量が足りなくなるたびに再確保とコピー
- 1要素ずつ汎用的なプロパティ書き込みを呼ぶ
- サブ配列ごとに再帰呼び出し

最適化前

空の配列から開始

ソース配列 6個のサブ配列、各10要素



結果配列のbacking store 容量 0

確保 0回 要素の書き込み 0回 再確保でのコピー 0要素 再帰呼び出し 0回

- 容量は 17 → 43 → 82 と伸び、そのたびに全要素をコピー
- 60要素に対して、確保3回とコピー60要素

ElementsKindとは

V8は配列の中身を

「整数だけ」「浮動小数点を含む」「Holeがある」といった
ElementsKindと呼ばれる内部型で追跡しており、

この型情報を使うと

「数値しか入っていない配列にはサブ配列が存在し得ない」ことが自明になり、
走査そのものを省略できます。

主要なElementsKind

	Packed (holeなし)	Holey (holeあり)
小さな整数 (Smi) だけ	PACKED_SMI_ELEMENTS	HOLEY_SMI_ELEMENTS
浮動小数点数を含む	PACKED_DOUBLE_ELEMENTS	HOLEY_DOUBLE_ELEMENTS
文字列やオブジェクトを含む	PACKED_ELEMENTS	HOLEY_ELEMENTS

遷移は原則一方向で、一度HOLEYになると
holeを埋めてもPACKEDには基本的に戻りません。

`PACKED_SMI_ELEMENTS` と `PACKED_DOUBLE_ELEMENTS` には
数値しか入りません。

- サブ配列は入らない
- Proxyも入らない
- holeもない

この保証を最適化に使うのが今回のパッチの肝です。

メモリ割り当てを
1回に減らす2パス方式

最適化の戦略

V8もJSCと同様に2パス方式を採用しました。

1. 第1パスで、結果配列の正確な長さとElementsKindを計算
2. その長さで、メモリを1回だけ確保
3. 第2パスで、要素を直接書き込む

参考にさせて頂いたSosuke Suzukiさんに感謝申し上げます。

<https://github.com/WebKit/WebKit/pull/56035>

60要素を平坦化するときの確保

最適化前



容量17を確保



容量43で確保し直し、17要素をコピー



容量82で確保し直し、43要素をコピー

2パス方式



長さ60で1回だけ確保

2パス方式

第1パス 長さを計算

ソース配列 6個のサブ配列、各10要素



第1パスで数えた長さ 0

確保 0回 要素の書き込み 0回 要素ごとの確認 0回

- 第1パスはサブ配列の `.length` を足すだけ
- 確保は1回、再確保に伴うコピーはなし

メモリ割り当て

$O(\log n)$ 回 → 1回

要素書き込みコスト

直接書き込みでオーバーヘッド激減

ElementsKind遷移

ゼロ

数値配列のショートカット

- ソースが `PACKED_SMI_ELEMENTS` か `PACKED_DOUBLE_ELEMENTS`
- 中身は数値だけで、holeもない
- だから要素を走査せず、長さはソースの `.length` そのまま

```
// src/builtins/array-flat.tq
if (sourceKind == ElementsKind::PACKED_SMI_ELEMENTS ||
    sourceKind == ElementsKind::PACKED_DOUBLE_ELEMENTS) {
  return FlattenedLengthResult{length: sourceLength, targetKind: sourceKind};
}
```

`[1, 2, 3, ..., 1024].flat()` の長さ計算は $O(1)$ になります。

サブ配列に対するショートカット

数値だけのサブ配列は、中を読まずに `.length` を足します。

```
// src/builtins/array-flat.tq
if (rawKind == ElementsKind::PACKED_SMI_ELEMENTS) {
  seenSmi = true;
  const subLen: Smi =
    Cast<Smi>(elementArray.length) otherwise goto Bailout;
  targetLength =
    math::TrySmiAdd(targetLength, subLen) otherwise goto Bailout;
  index++;
  continue;
}
```

```
[[1,2,3], [4,5,6], [7,8,9]].flat()
```

外側の配列 `PACKED_ELEMENTS`



結果の長さ 9

- 走査するのは外側の3要素だけ
- サブ配列の中身は読まない

ElementsKindの追跡

長さを数えながら、結果配列に入る値の型を3つのフラグで追跡します。

```
// src/builtins/array-flat.tq
if (!IsNumber(element)) {
  seenObject = true;
} else if (!TaggedIsSmi(element)) {
  seenDouble = true;
} else {
  seenSmi = true;
}
```

走査が終わったら、フラグから結果配列のElementsKindを決めます。

```
// src/builtins/array-flat.tq
if (seenObject) {
  targetKind = ElementsKind::PACKED_ELEMENTS;
} else if (seenDouble) {
  targetKind = ElementsKind::PACKED_DOUBLE_ELEMENTS;
} else {
  targetKind = ElementsKind::PACKED_SMI_ELEMENTS;
}
```

`[[1], [2], [3]].flat()` の結果は `PACKED_SMI_ELEMENTS` になります。

型が混ざる配列の平坦化

最適化前



2パス方式



- 要素を足すたびのElementsKindの遷移が起きない

SmiとDoubleが混ざると `PACKED_DOUBLE_ELEMENTS` になります。

```
// test/mjsunit/array-flat-elements-kind.js
assertSmiElementsKind([[1],[1]].flat());
assertDoubleElementsKind([[1],[1.1]].flat());
assertObjectElementsKind(["hello"].flat());
```

SmiはDoubleで表せるので、
汎用の `PACKED_ELEMENTS` まで広げずに済みます。

flatの後に続く処理への影響

- V8のJITコンパイラは、ElementsKindに合わせて特殊化したコードを生成
- 結果が `PACKED_SMI_ELEMENTS` なら、後段のループで型チェックを省ける

holeを詰める
flat特有の仕様との格闘

```
> [1, , 3].flat()
[ 1, 3 ]
> [1, , 3].map(n => n * 2)
[ 2, <1 empty item>, 6 ]
```

- `map` はholeを残す
- `flat` はholeを飛ばして詰める

最適化するうえで厄介な点

ソース `.length` は 5



↓ `flat()`

結果 長さは 3



- `.length` と結果の要素数が一致しない
- 正確な長さを先に出すには、holeを数える必要がある

HOLEY配列の扱い

- HOLEY配列には `.length` のショートカットが使えない
- 要素を1つずつ走査し、holeを除いて数える

```
// src/builtins/array-flat.tq
let element: JSAny;
try {
  element = fastOW.LoadElementNoHole(index) otherwise FoundHole;
} label FoundHole {
  index++;
  continue;
}
```

- `LoadElementNoHole` はholeを見つけるとラベルへジャンプ
- そのインデックスを飛ばすので、結果は詰まった配列になる
- HOLEY配列でも、1回だけ確保する恩恵は受けられる

V8独自言語Torqueでの実装

Torque

V8の組み込み関数を書くための専用言語

fast path / slow path

速いが条件付きの経路と、どんな入力でも正しく処理できる経路

bailout

fast pathの前提が崩れたとき、slow pathへ退避すること

- 従来の `flat` は、Torqueの `FlattenIntoArrayFast` と `FlattenIntoArraySlow`
- その手前に `TryFastFlat` を追加

```
// src/builtins/array-flat.tq
try {
  return TryFastFlat(o, len, depthSmi) otherwise SlowFastPath;
} label SlowFastPath {}

const a: JSReceiver = ArraySpeciesCreate(context, o, 0);
```

条件を満たさなければ、従来のslow pathへフォールバックします。

フォールバックする入力

- `Symbol.species` がオーバーライドされている
- ソース配列がProxy
- サブ配列にProxyが含まれている
- 配列がFastモードでない (dictionary modeなど)
- 長さがSmiの範囲を超える
- ネストが深すぎる

明示的スタックによる反復処理

- 再帰で書くと生成コードが複雑になりすぎて、buildが失敗
- 明示的なスタックを使った反復処理に変更

```
// src/builtins/array-flat.tq  
if (stack.length >= kMaxFlatFastStackEntries) goto Bailout;  
stack.Push(currentArray);  
stack.Push(nextIndex);  
stack.Push(currentDepth);
```

- 1エントリは、配列参照、インデックス、深さの3要素
- 上限は3072エントリで、深さ1024まで

安全性の担保と Bailout

fast pathは「楽観的だが安全」に作ります。

- 前提が揃っていれば高速なコードを実行
- 前提を満たさなくなったらslow pathへフォールバック

bailoutする場所

場所	確認すること
fast pathの入口	長さが S_{mi} の範囲内か、配列がfast modeか
走査中	<code>Recheck()</code> で配列の構造が変わっていないか
第2パスの最後	書き込んだ要素数と、第1パスで数えた長さが一致するか

ベンチマーク結果

- d8で計測、20,000個 × 1,024要素で合計約20M要素
- depth=1、50回計測の中央値

配列型	パッチ適用 (median)	main (median)	改善倍率
SMI (整数)	39.32 ms	181.06 ms	~4.6x
DOUBLE (浮動小数点数)	48.21 ms	224.80 ms	~4.7x
OBJECT (文字列)	79.56 ms	190.80 ms	~2.4x

2パス方式で最大約5倍になった `flat` を、
サブ配列の「バルクコピー」で
さらに約5倍速くしました。

2つを合わせると、手を入れる前と比べて約20倍です。

どこがまだ遅かったのか

- 2パス方式でも、第2パスのコピーは1個ずつ
- [1, 2, 3] はメモリ上に整数が3つ並んでいるだけ
- 本来はブロックごと移せば済む

1要素ごとに走る4つの確認

- 配列の構造が変わっていないかの再確認 (Recheck)
- holeではないか
- Proxyではないか
- 書き込み先があふれていないか

コピー本体より、この付帯チェックのほうが重いくらいです。

数値サブ配列のバルクコピー

数値だけのPackedなサブ配列は、backing storeをまるごとコピーします。

サブ配列 [1, 2, 3] のメモリ



↓ memcpy で長さ分を一括コピー

結果配列のメモリ



バルクコピー

第1パス 長さを計算

ソース配列 6個のサブ配列、各10要素



第1パスで数えた長さ 0

確保 0回 ブロックコピー 0回 型の確認 0回

- 60回の個別書き込みが、6回のブロックコピーに
- 4つの確認も、サブ配列1つにつき型の確認1回だけ

コピーには、最終的に `libc` の `memcpy` を呼ぶ `TorqueCopyElements` を使います。

```
// src/builtins/array-flat.tq
if (subArray.map.elements_kind == ElementsKind::PACKED_SMI_ELEMENTS) {
  const srcElements: FixedArray = Cast<FixedArray>(subArray.elements)
  otherwise goto Bailout;
  const srcLen: Smi = Cast<Smi>(subArray.length) otherwise goto Bailout;
  const newIdx: Smi = math::TrySmiAdd(targetIndex, srcLen)
  otherwise goto Bailout;
  if (Convert<intptr>(newIdx) > vector.fixedArray.length_intptr) {
    goto Bailout;
  }
  TorqueCopyElements(
    vector.fixedArray, SmiUntag(targetIndex), srcElements, 0,
    SmiUntag(srcLen));
  targetIndex = newIdx;
  index++;
  continue;
}
```

`PACKED_DOUBLE_ELEMENTS` も同じ要領で `FixedDoubleArray` をコピーします。

2パス方式があるから使える近道

- 第1パスで、コピー先の長さと型が確定している
- 配列を作り直す必要がない
- 要素ごとに型をそろえ直す必要もない

なぜ「数値配列だけ」 一括コピーできるのか

- 使えるのは `PACKED_SMI_ELEMENTS` と `PACKED_DOUBLE_ELEMENTS` だけ
- `PACKED_ELEMENTS` には使えない
- 境目は「write barrier」

write barrierとは何か

- V8のGCは世代別で、若い世代と古い世代を別々のタイミングで掃除する
- だからGCは、どのオブジェクトがどこを参照しているかを把握しておく必要がある
- ポインタを書き込むたびに「ここに参照ができた」とGCへ知らせる
- この通知がwrite barrier

なぜmemcpyだと危ないのか

- `memcpy` はビット列を複製するだけで、write barrierを発行しない
- 中身がポインタだと、GCが新しい参照に気づけない
- まだ使われているオブジェクトを回収し、use-after-freeでクラッシュやメモリ破壊

どの型なら安全か

ElementsKind	中身	ポインタを含む？	一括コピー
PACKED_SMI_ELEMENTS	小さな整数 (Smi)	含まない (値そのもの)	できる
PACKED_DOUBLE_ELEMENTS	生の浮動小数点数	含まない	できる
PACKED_ELEMENTS	文字列やオブジェクト	含む (ポインタの配列)	単純にはできない

hole埋めの省略

- `AllocateFixedDoubleArrayWithHoles` は、全スロットをholeを表す値で埋めてから返す
- 2パス方式なら、全スロットが第2パスで必ず埋まる
- このhole埋めは無駄
- 初期化しない `AllocateFixedArray` で確保し、`FixedDoubleArray` として扱う

	2パス方式	バルクコピー
確保	<code>AllocateFixedDoubleArrayWithHoles</code>	<code>AllocateFixedArray</code>
hole埋め	全スロットを初期化	しない
正しさ	常に安全	全スロットを必ず書くので安全

レビューでどう磨かれたか

- レビューは前回に引き続き Olivier Flückigerさん
- 最初にpushした実装から、だいぶ形が変わった

Recheckの巻き上げ

- `Recheck` は、配列の構造が変わっていないかの確認
- 最初の実装は、要素ごとに呼んでいた
- Olivierさん「これは巻き上げられるはず」
- 配列ごとに1回へ移動

```
while (true) {  
+  fastOW.Recheck() otherwise goto Bailout;  
  while (index < currentLength) {  
-   fastOW.Recheck() otherwise goto Bailout;  
    if (index >= fastOW.Get().length) goto Bailout;
```

巻き上げても安全な理由

- 内側ループの中ではJSのコードが動かない
- 構造が変わりうる処理は、内側ループを抜けるかbailoutする
- だから配列ごとに1回の確認で足りる

PACKED_ELEMENTSをどうするか

一番もめたのが、PACKED_ELEMENTS を一括コピーに含めるかどうかでした。

1. 最初は PACKED_SMI_ELEMENTS と PACKED_ELEMENTS の両方を一括コピー
2. Olivierさん「それはwrite barrierを飛ばすので安全でない」
3. PACKED_ELEMENTS はwrite barrier付きの1個ずつのストアに変更
4. Olivierさん「むしろそのケースは消した方がよい。利得もわずかに見える」

PACKED_ELEMENTSをどうするか

1. それでも手元では、ループ版が約30%速い
2. Olivierさん「思ったより大きいね。その時間はどこで使われているの？」
3. 正体は `memcpy` ではなく、4つの確認を省けた分
4. 速いケースを先に片づける構造案は、読みにくいので元に戻す

最終的に `memcpy` するのは数値配列の2種類だけになりました。

深さ制限の撤廃

- 最初の実装は、一括コピーを最も深い階層だけに限定
- Olivierさん「その限定も要らない」
- 数値だけのPacked配列はサブ配列を含まないので、深さに関係なくリーフ
- `[[1, 2, 3]].flat(5)` でも一括コピーできる

ベンチマーク

- d8 (arm64) で、V8 14.6.206とmainを比較
- 1,000個 × 1,000要素を flat(2)、8回計測の最小値

サブ配列の型	最適化前	最適化後	速度比
PACKED_SMI_ELEMENTS	6.09 ms	0.26 ms	約24x
PACKED_DOUBLE_ELEMENTS	7.56 ms	0.53 ms	約14x
PACKED_ELEMENTS	5.92 ms	3.87 ms	約1.5x
HOLEY_SMI_ELEMENTS	5.91 ms	3.45 ms	約1.7x
HOLEY_DOUBLE_ELEMENTS	7.49 ms	4.66 ms	約1.6x
HOLEY_ELEMENTS	6.12 ms	3.85 ms	約1.6x

3つの実装の比較

	最適化前	2パス方式	バルクコピー
結果配列の確保	足りなくなるたびに作り直し	長さを数えて1回だけ	1回だけ
サブ配列の処理	再帰で降りて1個ずつ追加	1個ずつ直接書き込み	数値配列は <code>memcpy</code> で一括
要素ごとの確認	あり	あり	数値配列ではなし
速度の目安	1x	約5x	約20x

- 大きく伸びたのは数値だけのPacked配列で、`PACKED_SMI_ELEMENTS` は約24倍
- 一括コピーの対象外の型も、1回確保のおかげで1.5倍ほど速い

V8に大きな変更を
入れるまでの流れ

- 提案 v8-devに実装方針を投稿
- 合意 Leszek Swirskiさん「良さそうだね。パッチ出して、議論はそっちでしょうか」
- レビュー GerritでLeszek SwirskiさんとOlivier Flückigerさんがレビュー
- マージ 最初のコミットから約1ヶ月でV8 14.7へ
- 翌日 ClusterFuzzがバグを3件発見

まずv8-devで提案する

- 大きな変更は、いきなりパッチを出さない
- 先にレビュアーと実装方針の合意を取るのが推奨
- v8-devに投稿すると、誰かしら反応してくれる

<https://groups.google.com/g/v8-dev/c/8ROaTLSDXkM>

最初のコミットから約1ヶ月、やりきりました。

[array] Add Torque fast path for Array.prototype.flat

Reviewed-on: <https://chromium-review.googlesource.com/c/v8/v8/+7526287>

Reviewed-by: Olivier Flückiger <olivf@chromium.org>

Reviewed-by: Leszek Swirski <leszeks@chromium.org>

Commit-Queue: Olivier Flückiger <olivf@chromium.org>

Commit-Queue: Leszek Swirski <leszeks@chromium.org>

Cr-Commit-Position: refs/heads/main@{#105498}

ClusterFuzzとの戦い

マージの翌日、GoogleのClusterFuzzが3件発見しました。

- crbug 488366773
- crbug 488586038
- crbug 489008235

原因は `GetPackedElementsKind` が

`HOLEY_DOUBLE_ELEMENTS` を `PACKED_DOUBLE_ELEMENTS` として扱っていたことです。

```
// src/builtins/array-flat.tq
macro GetPackedElementsKind(kind: ElementsKind): ElementsKind {
  if (kind == ElementsKind::HOLEY_SMI_ELEMENTS)
    return ElementsKind::PACKED_SMI_ELEMENTS;
  if (kind == ElementsKind::HOLEY_DOUBLE_ELEMENTS)
    return ElementsKind::PACKED_DOUBLE_ELEMENTS;
  if (kind == ElementsKind::HOLEY_ELEMENTS)
    return ElementsKind::PACKED_ELEMENTS;
  return kind;
}

const sourceKind: ElementsKind =
  GetPackedElementsKind(source.map.elements_kind);
```

なぜクラッシュしたのか

- `HOLEY_DOUBLE_ELEMENTS` はholeを含むので、`.length` と要素数が一致しない
- 第1パスの長さと、実際の要素数がずれる
- `V8_ENABLE_UNDEFINED_DOUBLE` が有効だと、`FixedDoubleArray`に`undefined`が入る
- 第2パスの `UnsafeCast<Number>` が`undefined`に当たってクラッシュ

```
// src/builtins/array-flat.tq  
doubleElements.values[targetIndex] =  
  Convert<float64_or_undefined_or_hole>(UnsafeCast<Number>(element));
```

修正と Close

- `GetPackedElementsKind` を削除し、`source.map.elements_kind` を直接見る
- 本当にPACKEDなElementsKindだけをショートカットの対象に
- 回帰テストを足し、CLの `Bug:` にissue番号を書いてマージ
- ClusterFuzzが自動で再評価し、修正を確認するとClose

**ClusterFuzzが品質を守っていることを
実感した瞬間でした。**

まとめ

- ElementsKindを使い、数値だけのPacked配列は走査そのものを省略
- 長さを先に数えて、結果配列の確保を1回に
- 数値だけのサブ配列は、write barrierが要らないので `memcpy` で一括コピー
- 最初のコミットから約1ヶ月でマージ、翌日にClusterFuzzが3件を発見

参考資料

- X: x.com/bunjavascript/status/2011898197330075845
- WebKit: [WebKit/WebKit#56035](#)
- v8-dev: [\[Design/Perf\] C++ fast path for Array.prototype.flat \(2-pass, V8\)](#)
- Gerrit: [\[array\] Add Torque fast path for Array.prototype.flat](#)
- Gerrit: [\[array\] Fix flat fast path crash on HOLEY_DOUBLE with undefined](#)
- Gerrit: [\[array\] Add bulk copy shortcuts to Array.flat fast path](#)
- Zenn: [Chromium\(V8\)のArray.prototype.flatを最大約5倍高速化した](#)
- Zenn: [Chromium\(V8\)のArray.prototype.flatをバルクコピーでさらに約5倍高速化した](#)

ご清聴ありがとうございました